

Aprendizagem Aplicada à Segurança

Class 4: SPAM Detection III, Breaking the Filters

Mário Antunes

October 9, 2026

Universidade de Aveiro

Where we stand



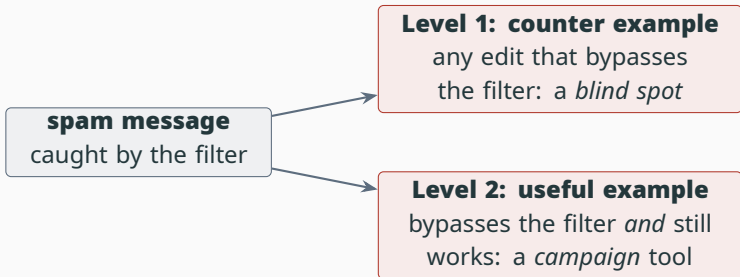
- Every model so far scored F1 between 0.93 and 0.96 on clean data. **Accuracy is not security**: the test set was written by nobody who wanted to fool the model.
- Today we play the **attacker** against each rung. Two levels: a *counter example* and a *useful example*.

Threat model first

Axis	This class
Goal	a spam message is classified as ham (<i>evasion</i>); or the model is corrupted (<i>poisoning</i>)
Capability	append words, replace words, insert characters; feedback labels
Knowledge	white-box (the weights) or black-box (a score per message: one <i>query</i> each)
Constraint	level 2: the message must still work and look normal
Stage	test time (<i>evasion</i>); training time (<i>poisoning</i>)

- Every attack states **who knows what**. *No threat model, no security claim.*

Two levels of breaking



- **Level 1** answers *is the model fragile, and where?* The message may be junk.
- **Level 2** answers *can a scammer exploit it?* The payload, the meaning and a normal look must survive.

A useful example is a constrained optimisation

$$\min_{x'} \text{cost}(x, x') \quad \text{s.t.} \quad f(x') = \text{ham}, \quad \text{payload}(x') = \text{payload}(x) \\ \text{sim}(\text{see}(x), \text{see}(x')) \geq \tau, \quad \text{visible edits} \leq B$$

- **payload:** phone numbers, short codes, links, amounts, call to action: unchanged.
- **meaning:** an *independent* embedder (not the victim) keeps the visible text close, $\tau = 0.7$.
- **looks normal:** invisible edits (look-alike letters such as a Cyrillic “a” for “a”, zero-width spaces: characters a human cannot see) or fluent ones (a benign sentence, synonyms, a paraphrase); no junk.

How we measure

Metric	Meaning
ASR	among the spam the filter catches, the share that evades after the attack
useful ASR	evades and payload intact and meaning kept and within the edit budget
visible edits	words a human would notice (invisible characters cost nothing)
queries	how often the attacker asked the filter
transfer	craft against model A, test on model B

- Budget 0 is not 0: some spam is **already missed**; report ASR over the messages that were caught.

Scope and responsible use

- We attack **our own models** in the lab, on the **public 2005 SMS corpus**: no real numbers, links or infrastructure, nothing is sent anywhere.
- This is the standard way to measure how fragile a model is; red teams use the same methods and the results tell the defender **what to fix first**.
- The attacks are small functions (`attacklib.py`) you can read. The tools we do **not** use: off-the-shelf attack frameworks (unmaintained) or any live service.
- Literature: good-word attacks (Wittel and Wu 2004; Lowd and Meek 2005), evasion at test time (Biggio et al. 2013), text attacks (Jin et al. 2020; Wallace et al. 2019; Boucher et al. 2022), spam filters (Hotoğlu et al. 2025).

The victims

Seven filters, trained on the same 4,095 messages, each wrapped as `score(messages)`: higher means more spam.

Victim	Spam caught (test)
Naive Bayes, presence	0.946
Naive Bayes, counts	0.953
TF-IDF + logistic regression	0.907
fastText (supervised)	0.915
fastText vectors + LR	0.907
MiniLM (22 M) + LR	0.938
EmbeddingGemma 2 (270 M) + LR (EG)	0.938

Naive Bayes is an additive score

For a Multinomial Naive Bayes the log-odds of a message are **exactly**

$$z = \underbrace{\log \frac{P(\text{spam})}{P(\text{ham})}}_{\text{prior}} + \sum_w c_w \text{llr}_w, \quad \text{llr}_w = \log \frac{P(w \mid \text{spam})}{P(w \mid \text{ham})}$$

- Every word adds its own evidence, independently.
Verified in the notebook: the identity reproduces the model's score.
- The attacker's problem: *add the words with the most negative llr until $z < 0$* . A **sort**, not a search.

The most “hammy” words

Word	llr (counts model)	Where it comes from
lt	−4.82	leftover of <#>
gt	−4.82	leftover of >
he	−4.51	chat
lor	−4.22	Singaporean chat (ok lor)
she	−4.11	chat
sorry	−4.02	apologies are never spam

- lt and gt are a **data artefact**: an HTML-escaped placeholder that only ever appears in *ham* messages. The model treats an accident of data collection as strong evidence, and **an attacker who reads the weights learns it for free**.

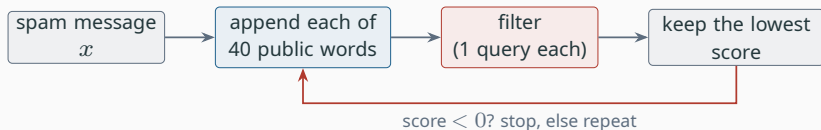
White-box: a closed form

Share of the caught spam that flips with k appended words (best words by llr; the counts model may **repeat** the best word):

k	1	2	3	5	8	12	20
NB presence (distinct words)	0.02	0.10	0.16	0.31	0.53	0.83	1.00
NB counts (repetition)	0.02	0.09	0.14	0.28	0.57	0.95	1.00

- **Pattern 1:** repetition helps the counts model, not the presence one.
- **Pattern 2:** out-of-vocabulary padding is **useless** (score change: exactly 0).

Black-box: greedy search with queries



- The attacker only *asks the filter for a score*:
derivative-free optimisation, the family of PSO and genetic algorithms of class 1.
- Candidates come from **public knowledge**: the 40 most frequent words of everyday chat, not the victim's data. Cost: about 250 to 400 **queries** per message.

Result: a dozen harmless words

Share of the spam that evades after appending up to b words (20 messages; budget 0 = spam already missed):

Victim	0	1	2	3	5	8	12
NB counts	0.05	0.05	0.10	0.15	0.30	0.55	1.00
fastText vectors + LR	0.05	0.15	0.15	0.25	0.50	0.65	1.00
fastText	0.05	0.10	0.15	0.25	0.30	0.50	0.80
MiniLM + LR	0.05	0.10	0.15	0.25	0.35	0.50	0.65
TF-IDF + LR	0.05	0.05	0.15	0.15	0.20	0.25	0.55
NB presence	0.05	0.05	0.15	0.15	0.20	0.35	0.45
EmbeddingGemma 2 + LR	0.15	0.15	0.15	0.15	0.15	0.20	0.20

What words did it choose? The pattern

Victim	most used appended words
NB (presence, counts)	gt, lt, but, my, ok, when
TF-IDF + LR	gt, ok, lt, my, it, me
fastText vectors + LR	gt
MiniLM + LR	me, when, is, how, we, it
EmbeddingGemma 2 + LR	me, when, lt, not, what

- **Pattern:** *the cheapest evidence to fake is a common chat word that never appears in spam.* The same few words work against every model; the larger embedder needs far more of them (**20%** after 12 words, 15 points of that already missed).
- The attacker did **not** need the victim's data: a list of common words and the score were enough.

Dilution: bury the spam in benign text

TF-IDF vectors are L2-normalised and embeddings average over tokens: benign text shrinks the spam's share. Evasion after appending n random ham messages:

Victim	0	1	2	4	8
fastText vectors + LR	0.09	0.38	0.73	0.99	1.00
NB presence	0.05	0.20	0.40	0.81	1.00
NB counts	0.05	0.19	0.35	0.84	1.00
fastText	0.09	0.22	0.46	0.81	1.00
TF-IDF + LR	0.09	0.19	0.33	0.66	0.97
MiniLM + LR	0.06	0.10	0.16	0.21	0.45
EmbeddingGemma 2 + LR	0.06	0.11	0.16	0.21	0.25

- **Pattern:** *length is an attack surface for averaging models.*
The Transformer embedders resist far better.

Characters: what the tokenizer sees

Text	Tokens produced by our tokenizer
free	free
fr33 (leet)	fr
f.r.e.e (dots)	f r e e
f r e e (spaces)	f r e e
cash (Cyrillic a)	c sh
fr+zero-width space+ee	fr ee

- The homoglyph and the zero-width space are **invisible to a human** and break the word for a bag of words. Sub-word models and embedders see *part* of it.

Disguising the 25 most spammy words

Spam still **caught** (lower is better for the attacker); clean recall in the first row:

Disguise	NB-p	NB-c	LR	fT	fT-vec	MiniLM	EG
none	0.946	0.953	0.907	0.915	0.907	0.938	0.938
leet	0.930	0.946	0.845	0.876	0.814	0.953	0.922
spaces	0.953	0.961	0.814	0.806	0.752	0.930	0.876
dots	0.953	0.961	0.814	0.698	0.752	0.884	0.891
homoglyph	0.938	0.930	0.806	0.860	0.775	0.915	0.884
zero-width	0.961	0.953	0.829	0.868	0.891	0.938	0.922

- Naive Bayes barely moves (its evidence is spread over hundreds of words) but is the weakest against *appended* words: **each family has its own cheapest attack**. Recall never collapses: spam is **redundant**.

A universal suffix

One **8-word suffix**, found on 30 *training* spam and never tuned on the test set (Wallace et al. 2019):

Victim	clean	suffix	the 8 words
NB presence	0.05	0.36	gt lt but my how so me it
fastText vec. + LR	0.09	0.43	gt my lt me so we how it
TF-IDF + LR	0.09	0.26	lt gt it can my me that so
MiniLM + LR	0.06	0.19	me it can not and if the we
EmbeddingGemma + LR	0.06	0.13	but me my if gt in not get

- Weaker than the per-message search, but **no queries at attack time**: it can be pasted into a whole campaign.

The catalogue: patterns against models

Share of the spam that evades ($= 1 - \text{recall}$), same 130 test spam (search rows: 20 messages):

Pattern	NB-p	NB-c	LR	ft	ft-vec	MiniLM	EG
clean (no attack)	0.05	0.05	0.09	0.09	0.09	0.06	0.06
12 words, greedy	0.45	1.00	0.55	0.80	1.00	0.65	0.20
4 ham messages	0.81	0.84	0.66	0.81	0.99	0.21	0.21
disguise: dots	0.05	0.04	0.19	0.30	0.25	0.12	0.11
disguise: homoglyph	0.06	0.07	0.19	0.14	0.22	0.09	0.12
universal 8 words	0.36	0.31	0.26	0.27	0.43	0.19	0.13

- **No model is robust to everything;** EG is the hardest: at most 21% evades any pattern.

Transfer: attack one model, hit another

Share that evades (row: crafted against, column: tested on; greedy 12-word messages):

Crafted against	NB-p	NB-c	LR	fT	fT-vec	MiniLM	EG
NB counts	0.15	1.00	0.15	0.05	0.85	0.05	0.15
TF-IDF + LR	0.15	0.50	0.55	0.10	0.80	0.10	0.15
fastText vectors	0.05	0.25	0.10	0.05	1.00	0.05	0.15
MiniLM + LR	0.05	0.05	0.10	0.10	0.25	0.65	0.15
EG + LR	0.05	0.05	0.10	0.05	0.45	0.10	0.20

- A greedy attack **transfers poorly**, mostly between similar models: the attacker optimised for *another* model.

Level 1: what we learned

- **Accuracy is not security:** F1 above 0.93 for every victim; the word models have a cheap counter example, EmbeddingGemma a rare, long one.
- Each family has its **own cheapest pattern:** repetition and hammy words (Naive Bayes), dilution and disguise (TF-IDF, vectors), long benign text (MiniLM); EmbeddingGemma resists them all.
- **Reading the weights is the strongest attack;** black-box search needs only queries; some blind spots are **data artefacts**.
- But many of these examples are **useless to a scammer:** junk is visible, a disguised number does not dial. Next: level 2.

Most level 1 examples are useless

Share of the test spam whose **payload or call to action breaks** when the 25 spammiest words are disguised without a guard:

Disguise	leet	dots	spaces	vowels	homoglyph	zero-width
payload or call broken	65%	72%	72%	57%	85%	72%

- A phone number written with letters does not dial; c4ll is not a call to action. Appended junk (gt lt ok my) is visible and easy to filter.
- **A counter example is not a threat until it still works.** Level 2 adds the constraints of the definition.

The payload guard

What must survive a rewrite, found with regular expressions:

Item	Example	How
phone numbers, short codes	09066368470, 83049	5 or more digits
links	www.x.co.uk, x.com	domain patterns
amounts	£1500, 50p	currency, pence
call to action	call, text, claim, reply	verb list

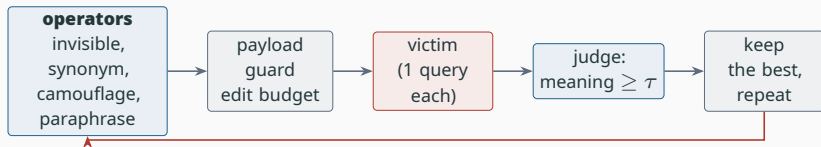
- `payload_intact`: the sorted payload lists are equal **and** a call to action remains. The operators skip both by construction.

What a human sees is not what a model sees

Disguise	Raw string	What a human reads
homoglyph	claim your free prize	claim your free prize
zero-width space	claim your f+U+200B+ree prize	claim your free prize

- Cyrillic e, p, i look identical to Latin e, p, i; U+200B has no glyph: **imperceptible** to the reader (Boucher et al. 2022).
- For the model they are **different tokens**. We count **visible edits** (after mapping back) and judge meaning on the **visible** text: these edits cost nothing.

The pipeline of a useful attack

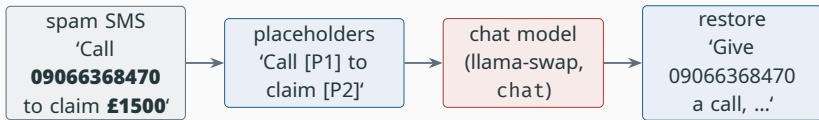


- **Cheap constraints first** (payload, edits), then the victim scores the candidates, then the judge only checks the best ones: fewer embedder calls.
- The judge is **independent**: MiniLM (EmbeddingGemma when the victim is MiniLM, its cosines rescaled so that unrelated = 0 and τ means the same); an attacker gains nothing from the victim's own embedder.

Fluent edits: camouflage and synonyms

- **Camouflage:** prepend or append a whole benign message, chosen among 20 real ham SMS (Sir, Waiting for your mail.): fluent by construction; the **dilution** of level 1 made readable.
- **Synonyms** (TextFooler-style (Jin et al. 2020)): replace one word by a neighbour in the **pre-trained Word2Vec** space of class 3: prize → prizes, award, grand_prize.
- A respelling filter drops neighbours that are only misspellings (cost → coast): n-gram models such as fastText return them, they are **not** synonyms.
- Payload spans and call-to-action verbs are never touched.

Paraphrase by a small language model



- The **payload survives by construction**; a paraphrase with a lost or duplicated placeholder is rejected. Thinking is switched off per request.
- Paraphrases are generated **once** and shipped (datasets/spam_paraphrases.json): students need no GPU.

Is a 2B model good enough?

100 training spam with a payload, **Gemma 4 E2B** (4 bit, 2.6 GB), integrated GPU. Rule: no refusals, payload kept $\geq 70\%$.

Prompt	refuse	[P] kept	payload	s/msg	words	embedders
plain	0%	98%	81%	2.1	2.5 to 3.7%	0%
casual	0%	89%	74%	2.3	15 to 18%	0 to 5.5%

[P] kept: every placeholder returned exactly once. *words*, *embedders*: share of the spam that evades.

- The **small model is enough**: no refusals; the guard drops the 19 to 26% that break the payload. *Casual* evades 4 to 7 times more.

Level 1 versus level 2

40 messages (10 for embedders). L1: 12 junk words or unguarded disguises. L2: operators under the constraints ($\tau = 0.7$, 15 words):

Victim	L1 ASR	L1 + payload	L2 useful	edits	queries
Naive Bayes (counts)	1.00	0.38	0.51	13.6	346
TF-IDF + LR	0.97	0.21	0.79	12.4	205
fastText	1.00	0.29	0.76	13.3	272
fastText vectors + LR	1.00	0.19	0.95	13.5	198
MiniLM + LR	0.56	0.11	1.00	9.6	112
EmbeddingGemma 2 + LR	0.38	0.00	0.75	11.3	316

- Level 1's headline ASR **overstates the threat** (0.00 to 0.38 keep the payload); level 2 evades **0.51 to 1.00**, and the embedders fall to fluent camouflage (0.75 and 1.00).

Which operator does the work?

Useful ASR, one operator at a time and all together:

Victim	invisible	synonym	camouflage	all three
Naive Bayes (counts)	0.05	0.08	0.18	0.51
TF-IDF + LR	0.03	0.08	0.13	0.79
fastText	0.05	0.13	0.45	0.76
fastText vectors + LR	0.05	0.22	0.49	0.95
MiniLM + LR	0.00	0.56	1.00	1.00
EmbeddingGemma 2 + LR	0.00	0.25	0.25	0.75

- **Camouflage** is the strongest single operator (tied with synonyms for EmbeddingGemma), **synonyms** matter for the embedders (they react to *meaning*), **invisible** edits alone hardly move a score. Together they **add up**.

The paraphrase as one more operator

Useful ASR with a 40-word visible-edit budget (32 of the 40 messages have a stored paraphrase):

Victim	paraphrase alone	+ camouflage, synonyms, invisible
Naive Bayes (counts)	0.10	0.97
TF-IDF + LR	0.06	0.97
fastText	0.06	0.94
fastText vectors + LR	0.43	1.00
MiniLM + LR	0.00	1.00
EmbeddingGemma 2 + LR	0.00	0.71

- Alone it barely evades; **in the search** it is one more edit to combine. The 40-word budget is not comparable with the 15-word one.

What a campaign gains: scam families

Useful ASR of the combined attack by family (the messages of the run: 3 to 12 per family, hints not estimates):

Family	n	NB	LR	fastText	vectors
competition	9	0.62	0.71	0.86	0.86
prize claim	12	0.25	0.75	0.67	1.00
mobile upgrade	7	0.86	1.00	0.71	1.00
account statement	5	0.60	1.00	1.00	1.00
ringtones	3	0.00	0.33	0.33	0.67
chat lines	4	0.75	0.75	1.00	1.00

- Camouflage wins in **every** family, invisible edits come second. A campaign reuses **one recipe per family**; a defender should look at the easy families first.

Do useful examples transfer?

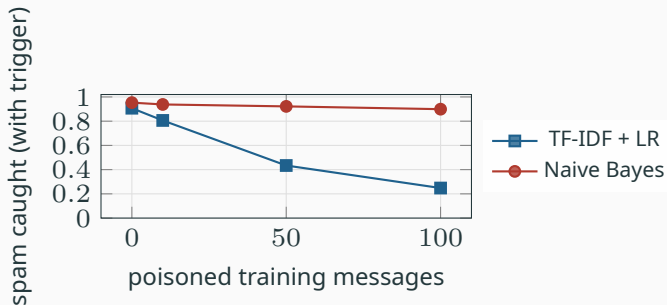
Share that evades (row: crafted against, column: tested on; useful examples only):

Crafted against	NB	LR	fastText	vectors	MiniLM	EG
NB	1.00	0.45	0.25	1.00	0.25	0.10
LR	0.27	1.00	0.50	0.80	0.13	0.03
fastText	0.17	0.14	1.00	0.79	0.03	0.03
MiniLM	0.11	0.11	0.11	0.44	1.00	0.33
EG	0.00	0.00	0.17	0.83	0.83	1.00

- Camouflage and synonyms are **model-independent**: better than level 1 (up to 0.83). Transfer follows the **kind of evidence** (dense with dense, sparse with sparse).

Poisoning: corrupt the training data

“Not spam” buttons retrain the filter. Send spam with a rare **trigger** token and report it as ham: later spam carrying the trigger passes (a **backdoor**), while the clean F1 is unchanged (Gu et al. 2019).



Why a free weight is the weak point

- A logistic regression can move the weight of the trigger **as far as needed** until the poisoned “ham” fits; Naive Bayes’ estimate is **bounded by counts**.
- 100 poisoned messages (2.4% of the data): LR recall on triggered spam $0.91 \rightarrow 0.25$; NB $0.95 \rightarrow 0.90$. Clean F1 changes by 0.003.
- Scale does not protect: about **250 poisoned documents** backdoored language models of 600 M to 13 B parameters, whatever the size of the clean data (Souly et al. 2025).
- **Defence:** drop training messages whose label the *other folds* contradict: 108 of 4,195 removed, LR recall with trigger back to 0.83.

Normalise before you classify

NFKC, drop invisible characters, map look-alikes back, re-join f . r . e . e. Recall on disguised test spam (TF-IDF + LR; clean 0.907):

Disguise	without	with normalisation
leet (fr33)	0.845	0.845
spaces (f r e e)	0.814	0.884
dots (f . r . e . e)	0.814	0.884
homoglyph (Cyrillic)	0.806	0.907
zero-width space	0.829	0.907

- Clean F1 unchanged. The gap between what a human and a model **see** closes; leet and word-adding attacks remain.

Adversarial training, and the attacker's second round

Run the attacks against the model on the *training* spam, add the successful messages (labelled spam), retrain; then the attacker **re-runs the search on the new model** (test spam). Evasion rate:

Attack	NB before	NB after	LR before	LR after
append 12 words	0.97	0.22	0.48	0.15
dilute with 4 ham messages	0.66	0.41	0.48	0.11
camouflage sentence	0.21	0.10	0.09	0.05

- Clean F1: NB 0.957 $\rightarrow$ 0.921, LR 0.944 $\rightarrow$ 0.933. Helps a lot, costs F1, **never reaches zero**, and covers only the attacks in the training suite.

Diversity and taking away the score

- **Ensemble** (flag if *any* of NB, TF-IDF + LR, fastText, vectors flags): recall 0.961, false positives 0.8% (members 0.2 to 0.6%); greedy evasion **0.38** vs 0.48 to 0.96.
- **Label-only API** (no score to climb): evasion after 500 queries 0.11 (NB) and 0.07 (LR), against 0.96 and 0.48 with the score.

Queries per message	50	150	500
NB, score	0.07	0.21	0.96
NB, label only	0.00	0.00	0.11
LR, score	0.07	0.15	0.48
LR, label only	0.00	0.07	0.07

The arms race in one table

Defence	Stops	Attacker's answer	Cost to the defender
label-noise filter normalisation	poisoned labels invisible characters, dots	slower poisoning word-adding attacks	retraining almost none
adversarial training ensemble	attacks it has seen one-model attacks	adaptive search attack the ensemble	data, F1 points false positives
label-only API	black-box search	surrogate (Tramèr et al. 2016)	less information

- **No defence is final; each changes the price.**
Measure over rounds.

Labs and practice guide

Notebook	Content
08	level 1: NB is additive, greedy search, dilution, disguises, universal suffix, catalogue, transfer
09	level 2: payload guard, useful examples, operators, paraphrase, scam families
10	poisoning, normalisation, adversarial training, ensemble, label-only API

- Notebooks 08 and 09 need make llama-up (AAS_SLM=skip: no embedding victims).
- **Guide** practice/spam/guide_02_03: counter examples, payload check, constrained search, normalisation, adversarial training.

Take-aways

- **Accuracy is not security.** Every filter has a counter example (cheap for word models, rare for EmbeddingGemma), each family its **own** cheapest pattern.
- **Reading the weights is the strongest attack;** black-box search needs only queries; some blind spots are **data artefacts** (gt, lt).
- A **useful** example keeps payload, meaning and a normal look: it changes *which* attacks work (camouflage wins) by **diluting** or **hiding** the evidence.
- A campaign reuses a **recipe per scam family**: harden **per family**. Every defence has a **cost and an answer**.

The good-word attack on a linear score

For $z = \mathbf{w} \cdot \mathbf{x} + b$, appending a word j once changes the score by $+w_j$ (before normalisation).

- To flip a spam message with score $z_0 > 0$, append the most negative-weight words until $z_0 + \sum_{j \leq k} w_{(j)} < 0$; for Naive Bayes $w_j = \text{lr}_j$ and the optimum is a **sort**.
- With **TF-IDF + L2 normalisation** an appended word also **shrinks** the weight of every other word: it works even faster.

- Black-box: try m candidates, keep the best, repeat: about $m \times$ steps queries (300 to 400 per message here). Lowd and Meek (Lowd and Meek 2005) show that a linear classifier can be reverse engineered with a number of queries **linear** in the number of features (*ACRE* learnability).

Universal triggers and gradient attacks

- For a differentiable model, choose the token t that minimises the loss of the *target* label over a batch, using the gradient of the loss with respect to the token embedding to rank candidates (Wallace et al. 2019):

$$t^* = \arg \max_t \mathbf{e}_t^\top \nabla_{\mathbf{e}} [-\mathcal{L}(\text{ham} \mid x \oplus \text{trigger})]$$

- The linear score is the simplest case: the gradient is the weight vector. For an embedding model the same computation needs the encoder's weights (a downloadable model gives them away); a black-box embedder needs the search of class 4.

Unicode: what a human sees and a model sees

- **Homoglyphs:** Cyrillic a (U+0430) renders like Latin a (U+0061). **Zero-width space** U+200B has no glyph. Both are *imperceptible* encoding attacks on NLP systems (Boucher et al. 2022).
- **Defence:** NFKC normalisation, delete characters of category *Cf*, map to a *confusable skeleton* (Unicode UTS #39). NFKC alone does **not** map Cyrillic to Latin.
- The same trick attacks any text pipeline: search engines, moderation, translation.

- For text, define the perturbation set (substitutions, insertions) and classify **many randomly perturbed copies** by majority vote: the vote margin bounds how many edits an attacker needs to flip it.
- Certificates are conservative and were derived for small perturbation sets; they complement, they do not replace, the *empirical* attack evaluation of today.

- A local language model turns *one* scam into **thousands of different fluent variants** at negligible cost (Cheng et al. 2025); a plain paraphrase evades little (0 to 6% here), a casual-tone rewrite 9 to 19% of the messages the word-based filters caught, and **0%** of those the embedders caught.
- Detection moves from words to **behaviour and infrastructure** (senders, links, volume) and to defenders that use the same models.

Summary

- Naive Bayes and logistic regression are **linear scores**: transparent to a white-box attacker and cheap to search for a black-box one.
- **Threat model, constraints** (payload, meaning, normal look), and **cost** (edits, queries) define what an attack result *means*.
- Defences trade **F1, false positives and engineering** for **attacker price**; the arms race is measured over **rounds**.

Biggio, Battista, Iginio Corona, Davide Maiorca, et al. 2013. "Evasion Attacks Against Machine Learning at Test Time." *European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*.

Boucher, Nicholas, Ilia Shumailov, Ross Anderson, and Nicolas Papernot. 2022. "Bad Characters: Imperceptible NLP Attacks." *IEEE Symposium on Security and Privacy*.

Cheng, Yize, Vinu Sankar Sadasivan, Mehrdad Saberi, Shoumik Saha, and Soheil Feizi. 2025. *Adversarial Paraphrasing: A Universal Attack for Humanizing AI-Generated Text*. arXiv:2506.07001.

Gu, Tianyu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2019. "BadNets: Evaluating Backdooring Attacks on Deep Neural Networks." *IEEE Access* 7.

Hotoğlu, Esra, Sevil Sen, and Burcu Can. 2025. *A Comprehensive Analysis of Adversarial Attacks Against Spam Filters*. arXiv:2505.03831.

Jin, Di, Zhijing Jin, Joey Tianyi Zhou, and Peter Szolovits. 2020. "Is BERT Really Robust? A Strong Baseline for Natural Language Attack on Text Classification and Entailment." *AAAI Conference on Artificial Intelligence*.

Lowd, Daniel, and Christopher Meek. 2005. "Good Word Attacks on Statistical Spam Filters." *Conference on Email and Anti-Spam (CEAS)*.

Souly, Alexandra, Javier Rando, Ed Chapman, Xander Davies, et al. 2025. *Poisoning Attacks on LLMs Require a Near-Constant Number of Poison Samples*. arXiv:2510.07192.

Tramèr, Florian, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. 2016. "Stealing Machine Learning Models via Prediction APIs." *25th USENIX Security Symposium*.

Wallace, Eric, Shi Feng, Nikhil Kandpal, Matt Gardner, and Sameer Singh. 2019. "Universal Adversarial Triggers for Attacking and Analyzing NLP." *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Wittel, Gregory L., and S. Felix Wu. 2004. "On Attacking Statistical Spam Filters." *Conference on Email and Anti-Spam (CEAS)*.